Results

Study 1: Feasibility of Predicting Keep/Remove Decisions

Motivation

... In Study 1, we asked users ...

Formulating the task

...

Dataset

Prolific participants (n=1,321) reviewed pairs of original and mirrored posts (n=959 pairs), resulting in a final dataset of n=13,250 rows, with each row representing a human keep/remove decision for the pair of posts. In these "linked fate" trials, the users saw both the original post and its mirrored equivalent, randomly ordered, and were asked to make a keep/remove decision for the pair. In this dataset, 66.2% of post pairs were kept (and 33.8% removed). We create training labels for each post by taking the modal label across all raters (average n=13.8 labels per post).

Training an initial set of models with explainable text features

We first generated a set of explainable text features:

- Does the text involve intergroup discussion?
- Does the text have PRIME content?
- Does the text have a positive valence?
- Does the text have toxic content?
- What is the political stance of the text?
- What is the number of characters in the text?
- What is the average sentence length?
- What is the Flesch-Kincaid reading score?

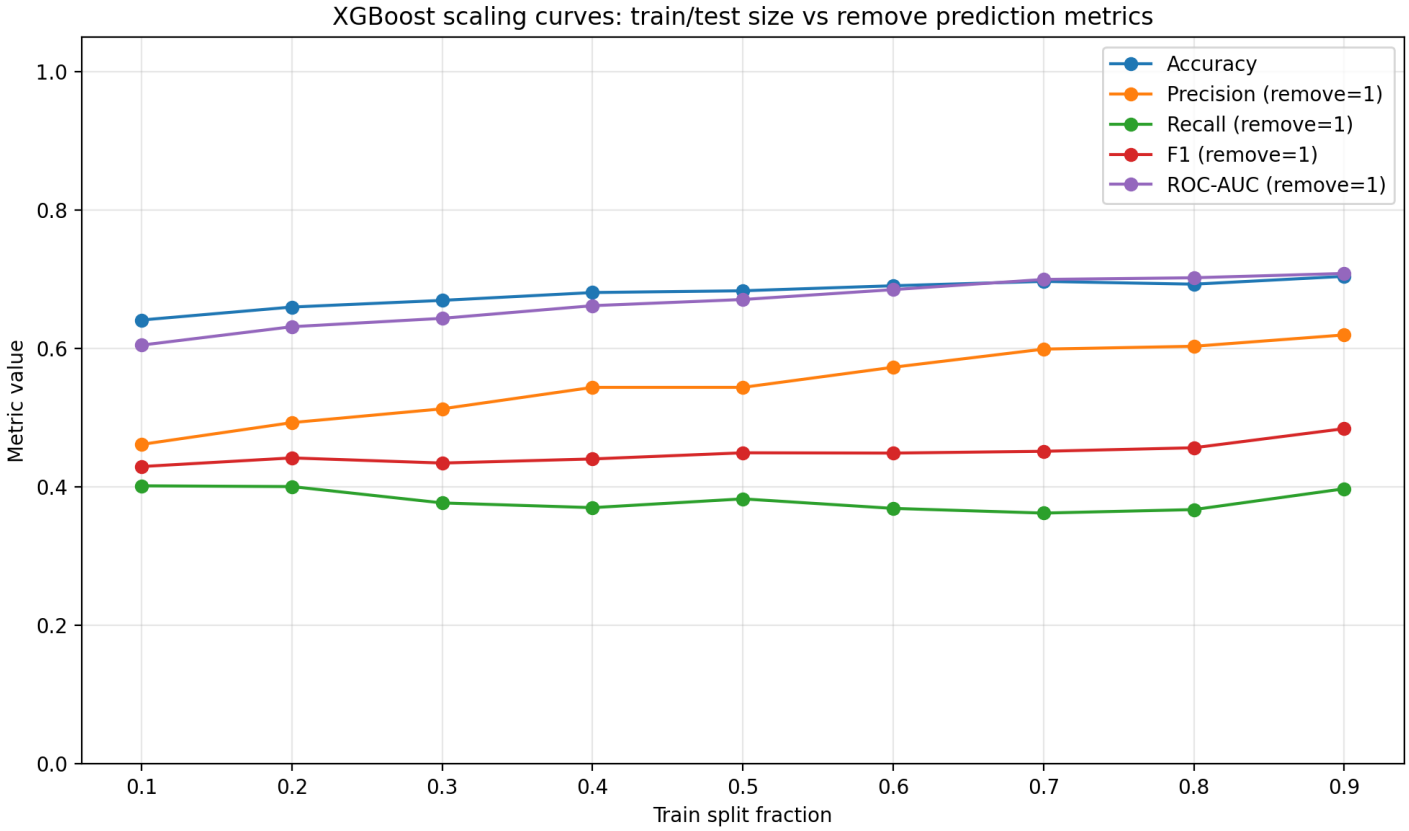
We then trained two models on these features: a logistic regression model and an XGBoost model. We report those results in a table

Caption: Model performance for predicting keep/remove decisions. Precision, recall, F1, and ROC-AUC are computed with the remove decision as the positive class ($y = 1$).

Model	Split	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline (always predict keep)	Test	0.649	0.000	0.000	0.000	0.500
Logistic regression	Train	0.699	0.572	0.407	0.475	0.675
Logistic regression	Test	0.695	0.597	0.402	0.480	0.678
XGBoost	Train	0.737	0.667	0.430	0.523	0.777
XGBoost	Test	0.693	0.603	0.367	0.457	0.702

We notice a slight lift in the logistic regression and XGBoost models as compared to a naive baseline, driven by improvements in recall of remove decisions. We also observe likely overfitting on the XGBoost model given the sparsity of our n=959 dataset.

Scaling curves We also generated scaling curves by varying the proportion of the n=959 posts used in the training vs. test sets.



From our scaling curves, we don't see much improvement in recall and we only see slight gains in precision and overall accuracy. We have a small dataset size overall so it's unlikely that we have the sample size to learn sufficient signal.

Training models based on the text embeddings

Dataset We also trained additional predictive models based on text embeddings. We used Amazon Bedrock's Titan Text Embeddings (`amazon.titan-embed-text-v2:0`), with `d=256` dimensions. The embeddings were also L2-normalized.

We generated the following vectors:

- The original embedding vector, with shape `(256,)`.
- The mirrored post's embedding vector, with shape `(256, )`.
- The elementwise absolute difference between the original and mirrored post's vectors, with shape `(256, )`.
- The Hadamard elementwise product between the original and mirrored post embedding vectors, with shape `(256, )`.
- The scalar cosine similarity between the original and mirrored posts, with shape `(1,)`.
- One-hot vector of the original post's political stance (left vs. right), with shape `(2,)`.
- One-hot vector of the original post's toxicity category (low, middle, high), with shape `(3,)`.

We then stack these vectors into a single feature vector with shape `(1030,)`, creating a dataset with shape `(959, 1030)` for our posts.

Results We show the model performance for predicting keep/remove decisions using Bedrock text embeddings. Precision, recall, F1, and ROC-AUC are computed with the remove decision as the positive class ($y = 1$).

Model	Split	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline (always predict keep)	Test	0.649	0.000	0.000	0.000	0.500
Logistic regression	Train	0.897	0.686	0.944	0.795	0.966
Logistic regression	Test	0.849	0.620	0.756	0.681	0.879
XGBoost	Train	1.000	1.000	1.000	1.000	1.000
XGBoost	Test	0.859	0.750	0.512	0.609	0.860

We observe that logistic regression generalizes better on the test set in terms of F1 and recall. XGBoost overfits more strongly,

which we believe to be a symptom of the relatively small sample size. However, overall, training on the text embeddings themselves was much more performant than training on the hand-crafted text features.

Test-set metric comparison (Accuracy / Precision / Recall / F1) Figure below compares the test-set metrics from the feature-engineered models vs. the text-embedding models, plotted on the same axes. Dashed lines correspond to models trained on feature engineering, while solid lines correspond to models trained on text embeddings. Line colors correspond to the model class (baseline vs. logistic regression vs. XGBoost).

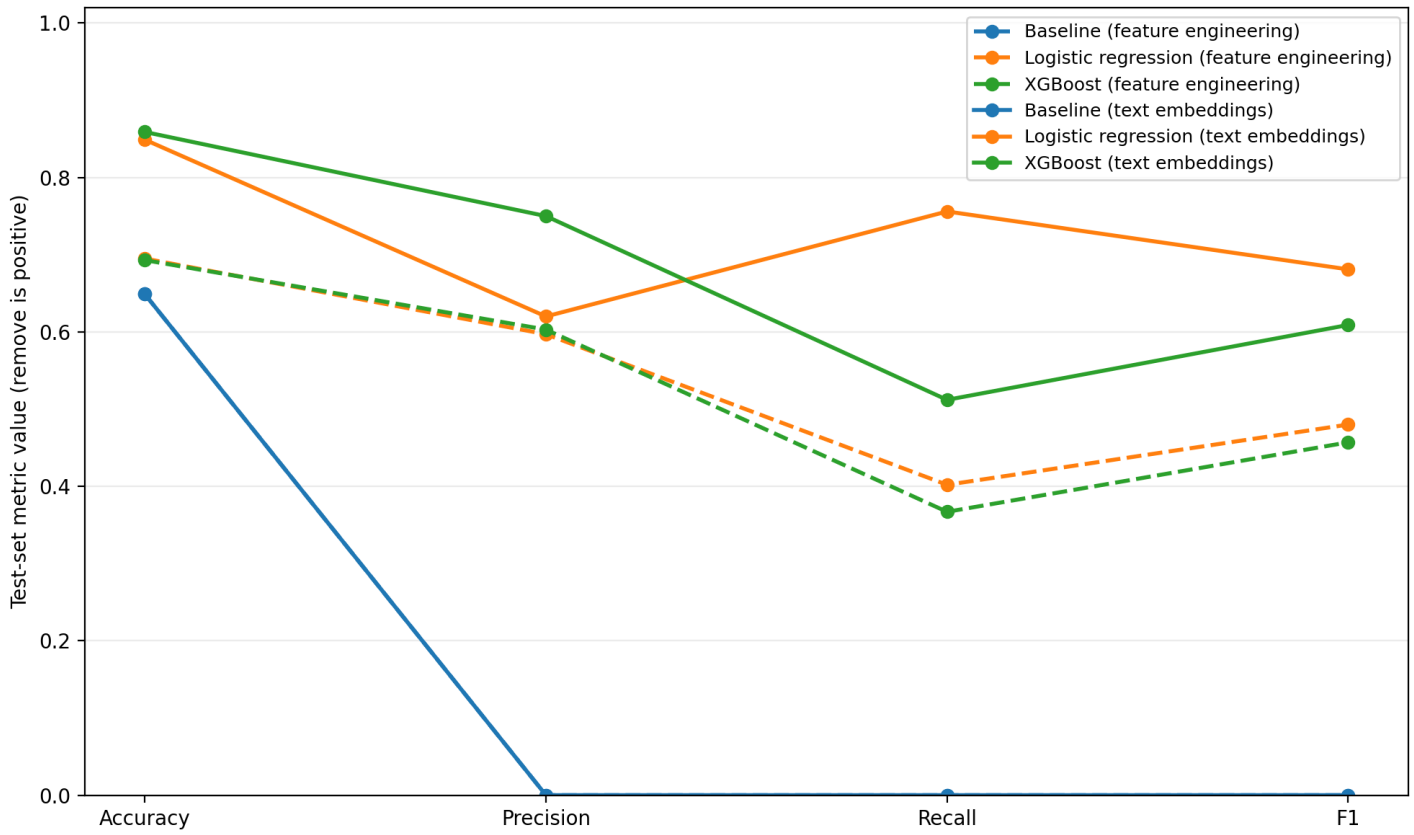


Figure 1: Test-set metric line graph

Study 2

In Study 2, we expanded on . . .

(initial results from Study 2)

Model performance

(model performance)